

Chapter 5. Dealing with Files

A lot of computer programming deals with files. After all, when we reboot our computers, the only thing that remains from previous sessions are the things that have been put on disk. Data which is stored in files is called *persistent* data, because it persists in files that remain on the disk even when the program isn't running..

The UNIX File Concept

Each operating system has it's own way of dealing with files. However, the UNIX method, which is used on Linux, is the simplest and most universal. UNIX files, no matter what program created them, can all be accessed as a sequential stream of bytes. When you access a file, you start by opening it by name. The operating system then gives you a number, called a *file descriptor*, which you use to refer to the file until you are through with it. You can then read and write to the file using its file descriptor. When you are done reading and writing, you then close the file, which then makes the file descriptor useless.

In our programs we will deal with files in the following ways:

1. Tell Linux the name of the file to open, and in what mode you want it opened (read, write, both read and write, create it if it doesn't exist, etc.). This is handled with the `open` system call, which takes a filename, a number representing the mode, and a permission set as its parameters. `%eax` will hold the system call number, which is 5. The address of the first character of the filename should be stored in `%ebx`. The read/write intentions, represented as a number, should be stored in `%ecx`. For now, use 0 for files you want to read from, and 03101 for files you want to write to (you must include the leading zero).¹ Finally, the permission set should be stored as a number in `%edx`. If

1. This will be explained in more detail in the Section called *Truth, Falsehood, and Binary Numbers* in Chapter 10.

Chapter 5. Dealing with Files

you are unfamiliar with UNIX permissions, just use 0666 for the permissions (again, you must include the leading zero).

2. Linux will then return to you a file descriptor in `%eax`. Remember, this is a number that you use to refer to this file throughout your program.
3. Next you will operate on the file doing reads and/or writes, each time giving Linux the file descriptor you want to use. `read` is system call 3, and to call it you need to have the file descriptor in `%ebx`, the address of a buffer for storing the data that is read in `%ecx`, and the size of the buffer in `%edx`. Buffers will be explained in the Section called *Buffers and .bss*. `read` will return with either the number of characters read from the file, or an error code. Error codes can be distinguished because they are always negative numbers (more information on negative numbers can be found in Chapter 10). `write` is system call 4, and it requires the same parameters as the `read` system call, except that the buffer should already be filled with the data to write out. The `write` system call will give back the number of bytes written in `%eax` or an error code.
4. When you are through with your files, you can then tell Linux to close them. Afterwards, your file descriptor is no longer valid. This is done using `close`, system call 6. The only parameter to `close` is the file descriptor, which is placed in `%ebx`

Buffers and .bss

In the previous section we mentioned buffers without explaining what they were. A buffer is a continuous block of bytes used for bulk data transfer. When you request to read a file, the operating system needs to have a place to store the data it reads. That place is called a buffer. Usually buffers are only used to store data temporarily, and it is then read from the buffers and converted to a form that is easier for the programs to handle. Our programs won't be complicated enough to need that done. For an example, let's say that you want to read in a single line of text from a file but you do not know how long that line is. You would then simply

read a large number of bytes/characters from the file into a buffer, look for the end-of-line character, and copy all of the characters to that end-of-line character to another location. If you didn't find an end-of-line character, you would allocate another buffer and continue reading. You would probably wind up with some characters left over in your buffer in this case, which you would use as the starting point when you next need data from the file.²

Another thing to note is that buffers are a fixed size, set by the programmer. So, if you want to read in data 500 bytes at a time, you send the `read` system call the address of a 500-byte unused location, and send it the number 500 so it knows how big it is. You can make it smaller or bigger, depending on your application's needs.

To create a buffer, you need to either reserve static or dynamic storage. Static storage is what we have talked about so far, storage locations declared using `.long` or `.byte` directives. Dynamic storage will be discussed in the Section called *Getting More Memory* in Chapter 9. There are problems, though, with declaring buffers using `.byte`. First, it is tedious to type. You would have to type 500 numbers after the `.byte` declaration, and they wouldn't be used for anything but to take up space. Second, it uses up space in the executable. In the examples we've used so far, it doesn't use up too much, but that can change in larger programs. If you want 500 bytes you have to type in 500 numbers and it wastes 500 bytes in the executable. There is a solution to both of these. So far, we have discussed two program sections, the `.text` and the `.data` sections. There is another section called the `.bss`. This section is like the `.data` section, except that it doesn't take up space in the executable. This section can reserve storage, but it can't initialize it. In the `.data` section, you could reserve storage and set it to an initial value. In the `.bss` section, you can't set an initial value. This is useful for buffers because we don't need to initialize them anyway, we just need to reserve storage. In order to do this, we do the following commands:

```
.section .bss
```

2. While this sounds complicated, most of the time in programming you will not need to deal directly with buffers and file descriptors. In Chapter 8 you will learn how to use existing code present in Linux to handle most of the complications of file input/output for you.

Chapter 5. Dealing with Files

```
.lcomm my_buffer, 500
```

This directive, `.lcomm`, will create a symbol, `my_buffer`, that refers to a 500-byte storage location that we can use as a buffer. We can then do the following, assuming we have opened a file for reading and have placed the file descriptor in `%ebx`:

```
movl $my_buffer, %ecx
movl 500, %edx
movl 3, %eax
int $0x80
```

This will read up to 500 bytes into our buffer. In this example, I placed a dollar sign in front of `my_buffer`. Remember that the reason for this is that without the dollar sign, `my_buffer` is treated as a memory location, and is accessed in direct addressing mode. The dollar sign switches it to immediate mode addressing, which actually loads the number represented by `my_buffer` (i.e. - the address of the start of our buffer). (which is the address of `my_buffer`) itself into `%ecx`.

Standard and Special Files

You might think that programs start without any files open by default. This is not true. Linux programs usually have at least three open file descriptors when they begin. They are:

STDIN

This is the *standard input*. It is a read-only file, and usually represents your keyboard.³ This is always file descriptor 0.

3. As we mentioned earlier, in Linux, almost everything is a "file". Your keyboard input is considered a file, and so is your screen display.

STDOUT

This is the *standard output*. It is a write-only file, and usually represents your screen display. This is always file descriptor 1.

STDERR

This is your *standard error*. It is a write-only file, and usually represents your screen display. Most regular processing output goes to STDOUT, but any error messages that come up in the process go to STDERR. This way, if you want to, you can split them up into separate places. This is always file descriptor 2.

Any of these "files" can be redirected from or to a real file, rather than a screen or a keyboard. This is outside the scope of this book, but any good book on the UNIX command-line will describe it in detail. The program itself does not even need to be aware of this indirection - it can just use the standard file descriptors as usual.

Notice that many of the files you write to aren't files at all. UNIX-based operating systems treat all input/output systems as files. Network connections are treated as files, your serial port is treated like a file, even your audio devices are treated as files. Communication between processes is usually done through special files called pipes. Some of these files have different methods of opening and creating them than regular files (i.e. - they don't use the `open` system call), but they can all be read from and written to using the standard `read` and `write` system calls.

Using Files in a Program

We are going to write a simple program to illustrate these concepts. The program will take two files, and read from one, convert all of its lower-case letters to upper-case, and write to the other file. Before we do so, let's think about what we need to do to get the job done:

- Have a function that takes a block of memory and converts it to upper-case. This function would need an address of a block of memory and its size as

Chapter 5. Dealing with Files

parameters.

- Have a section of code that repeatedly reads in to a buffer, calls our conversion function on the buffer, and then writes the buffer back out to the other file.
- Begin the program by opening the necessary files.

Notice that I've specified things in reverse order that they will be done. That's a useful trick in writing complex programs - first decide the meat of what is being done. In this case, it's converting blocks of characters to upper-case. Then, you think about what all needs to be setup and processed to get that to happen. In this case, you have to open files, and continually read and write blocks to disk. One of the keys of programming is continually breaking down problems into smaller and smaller chunks until it's small enough that you can easily solve the problem. Then you can build these chunks back up until you have a working program.⁴

You may have been thinking that you will never remember all of these numbers being thrown at you - the system call numbers, the interrupt number, etc. In this program we will also introduce a new directive, `.equ` which should help out. `.equ` allows you to assign names to numbers. For example, if you did `.equ LINUX_SYSCALL, 0x80`, any time after that you wrote `LINUX_SYSCALL`, the assembler would substitute `0x80` for that. So now, you can write

```
int $LINUX_SYSCALL
```

which is much easier to read, and much easier to remember. Coding is complex, but there are a lot of things we can do like this to make it easier.

Here is the program. Note that we have more labels than we actually use for jumps, because some of them are just there for clarity. Try to trace through the program and see what happens in various cases. An in-depth explanation of the program will follow.

#PURPOSE: This program converts an input file

4. Maureen Sprankle's *Problem Solving and Programming Concepts* is an excellent book on the problem-solving process applied to computer programming.

```
#           to an output file with all letters
#           converted to uppercase.
#
#PROCESSING: 1) Open the input file
#             2) Open the output file
#             4) While we're not at the end of the input file
#                 a) read part of file into our memory buffer
#                 b) go through each byte of memory
#                     if the byte is a lower-case letter,
#                     convert it to uppercase
#                 c) write the memory buffer to output file

.section .data

#####CONSTANTS#####

#system call numbers
.equ SYS_OPEN, 5
.equ SYS_WRITE, 4
.equ SYS_READ, 3
.equ SYS_CLOSE, 6
.equ SYS_EXIT, 1

#options for open (look at
#/usr/include/asm/fcntl.h for
#various values.  You can combine them
#by adding them or ORing them)
#This is discussed at greater length
#in "Counting Like a Computer"
.equ O_RDONLY, 0
.equ O_CREAT_WRONLY_TRUNC, 03101

#standard file descriptors
.equ STDIN, 0
.equ STDOUT, 1
```

Chapter 5. Dealing with Files

```
.equ STDERR, 2

#system call interrupt
.equ LINUX_SYSCALL, 0x80

.equ END_OF_FILE, 0  #This is the return value
                     #of read which means we've
                     #hit the end of the file

.equ NUMBER_ARGUMENTS, 2

.section .bss
#Buffer - this is where the data is loaded into
#         from the data file and written from
#         into the output file. This should
#         never exceed 16,000 for various
#         reasons.
.equ BUFFER_SIZE, 500
.lcomm BUFFER_DATA, BUFFER_SIZE

.section .text

#STACK POSITIONS
.equ ST_SIZE_RESERVE, 8
.equ ST_FD_IN, -4
.equ ST_FD_OUT, -8
.equ ST_ARGC, 0      #Number of arguments
.equ ST_ARGV_0, 4    #Name of program
.equ ST_ARGV_1, 8    #Input file name
.equ ST_ARGV_2, 12   #Output file name

.globl _start
_start:
###INITIALIZE PROGRAM###
#save the stack pointer
```



```
movl %esp, %ebp

#Allocate space for our file descriptors
#on the stack
subl $ST_SIZE_RESERVE, %esp

open_files:
open_fd_in:
    ###OPEN INPUT FILE###
    #open syscall
    movl $SYS_OPEN, %eax
    #input filename into %ebx
    movl ST_ARGV_1(%ebp), %ebx
    #read-only flag
    movl $O_RDONLY, %ecx
    #this doesn't really matter for reading
    movl $0666, %edx
    #call Linux
    int $LINUX_SYSCALL

store_fd_in:
    #save the given file descriptor
    movl %eax, ST_FD_IN(%ebp)

open_fd_out:
    ###OPEN OUTPUT FILE###
    #open the file
    movl $SYS_OPEN, %eax
    #output filename into %ebx
    movl ST_ARGV_2(%ebp), %ebx
    #flags for writing to the file
    movl $O_CREAT_WRONLY_TRUNC, %ecx
    #mode for new file (if it's created)
    movl $0666, %edx
    #call Linux
```

Chapter 5. Dealing with Files

```
int    $LINUX_SYSCALL

store_fd_out:
    #store the file descriptor here
    movl %eax, ST_FD_OUT(%ebp)

    ###BEGIN MAIN LOOP###
read_loop_begin:

    ###READ IN A BLOCK FROM THE INPUT FILE###
    movl $SYS_READ, %eax
    #get the input file descriptor
    movl ST_FD_IN(%ebp), %ebx
    #the location to read into
    movl $BUFFER_DATA, %ecx
    #the size of the buffer
    movl $BUFFER_SIZE, %edx
    #Size of buffer read is returned in %eax
    int    $LINUX_SYSCALL

    ###EXIT IF WE'VE REACHED THE END###
    #check for end of file marker
    cmpl $END_OF_FILE, %eax
    #if found or on error, go to the end
    jle    end_loop

continue_read_loop:
    ###CONVERT THE BLOCK TO UPPER CASE###
    pushl $BUFFER_DATA    #location of buffer
    pushl %eax            #size of the buffer
    call  convert_to_upper
    popl  %eax            #get the size back
    addl  $4, %esp        #restore %esp

    ###WRITE THE BLOCK OUT TO THE OUTPUT FILE###
```

```
#size of the buffer
movl %eax, %edx
movl $SYS_WRITE, %eax
#file to use
movl ST_FD_OUT(%ebp), %ebx
#location of the buffer
movl $BUFFER_DATA, %ecx
int $LINUX_SYSCALL

###CONTINUE THE LOOP###
jmp read_loop_begin

end_loop:
###CLOSE THE FILES###
#NOTE - we don't need to do error checking
#       on these, because error conditions
#       don't signify anything special here
movl $SYS_CLOSE, %eax
movl ST_FD_OUT(%ebp), %ebx
int $LINUX_SYSCALL

movl $SYS_CLOSE, %eax
movl ST_FD_IN(%ebp), %ebx
int $LINUX_SYSCALL

###EXIT###
movl $SYS_EXIT, %eax
movl $0, %ebx
int $LINUX_SYSCALL

#PURPOSE:   This function actually does the
#           conversion to upper case for a block
#
#INPUT:     The first parameter is the location
```

Chapter 5. Dealing with Files

```
#           of the block of memory to convert
#           The second parameter is the length of
#           that buffer
#
#OUTPUT:    This function overwrites the current
#           buffer with the upper-casified version.
#
#VARIABLES:
#           %eax - beginning of buffer
#           %ebx - length of buffer
#           %edi - current buffer offset
#           %cl  - current byte being examined
#                   (first part of %ecx)
#
#####CONSTANTS###
#The lower boundary of our search
.equ  LOWERCASE_A, 'a'
#The upper boundary of our search
.equ  LOWERCASE_Z, 'z'
#Conversion between upper and lower case
.equ  UPPER_CONVERSION, 'A' - 'a'
#####STACK STUFF###
.equ  ST_BUFFER_LEN, 8 #Length of buffer
.equ  ST_BUFFER, 12    #actual buffer
convert_to_upper:
    pushl %ebp
    movl  %esp, %ebp
#####SET UP VARIABLES###
    movl  ST_BUFFER(%ebp), %eax
    movl  ST_BUFFER_LEN(%ebp), %ebx
    movl  $0, %edi
```

```
#if a buffer with zero length was given
#to us, just leave
cmpl $0, %ebx
je   end_convert_loop

convert_loop:
#get the current byte
movb (%eax,%edi,1), %cl

#go to the next byte unless it is between
#a' and 'z'
cmpb $LOWERCASE_A, %cl
jl   next_byte
cmpb $LOWERCASE_Z, %cl
jg   next_byte

#otherwise convert the byte to uppercase
addb $UPPER_CONVERSION, %cl
#and store it back
movb %cl, (%eax,%edi,1)
next_byte:
incl %edi           #next byte
cmpl %edi, %ebx     #continue unless
                    #we've reached the
                    #end
jne  convert_loop

end_convert_loop:
#no return value, just leave
movl %ebp, %esp
popl %ebp
ret
```

Type in this program as `toupper.s`, and then enter in the following commands:

Chapter 5. Dealing with Files

```
as toupper.s -o toupper.o  
ld toupper.o -o toupper
```

This builds a program called `toupper`, which converts all of the lowercase characters in a file to uppercase. For example, to convert the file `toupper.s` to uppercase, type in the following command:

```
./toupper toupper.s toupper.uppercase
```

You will now find in the file `toupper.uppercase` an uppercase version of your original file.

Let's examine how the program works.

The first section of the program is marked `CONSTANTS`. In programming, a constant is a value that is assigned when a program assembles or compiles, and is never changed. I make a habit of placing all of my constants together at the beginning of the program. It's only necessary to declare them before you use them, but putting them all at the beginning makes them easy to find. Making them all upper-case makes it obvious in your program which values are constants and where to find them.⁵ In assembly language, we declare constants with the `.equ` directive as mentioned before. Here, we simply give names to all of the standard numbers we've used so far, like system call numbers, the `syscall` interrupt number, and file open options.

The next section is marked `BUFFERS`. We only use one buffer in this program, which we call `BUFFER_DATA`. We also define a constant, `BUFFER_SIZE`, which holds the size of the buffer. If we always refer to this constant rather than typing out the number 500 whenever we need to use the size of the buffer, if it later changes, we only need to modify this value, rather than having to go through the entire program and changing all of the values individually.

Instead of going on to the `_start` section of the program, go to the end where we define the `convert_to_upper` function. This is the part that actually does the

5. This is fairly standard practice among programmers in all languages.

conversion.

This section begins with a list of constants that we will use. The reason these are put here rather than at the top is that they only deal with this one function. We have these definitions:

```
.equ  LOWERCASE_A, 'a'
.equ  LOWERCASE_Z, 'z'
.equ  UPPER_CONVERSION, 'A' - 'a'
```

The first two simply define the letters that are the boundaries of what we are searching for. Remember that in the computer, letters are represented as numbers. Therefore, we can use LOWERCASE_A in comparisons, additions, subtractions, or anything else we can use numbers in. Also, notice we define the constant UPPER_CONVERSION. Since letters are represented as numbers, we can subtract them. Subtracting an upper-case letter from the same lower-case letter gives us how much we need to add to a lower-case letter to make it upper case. If that doesn't make sense, look at the ASCII code tables themselves (see Appendix D). You'll notice that the number for the character A is 65 and the character a is 97. The conversion factor is then -32. For any lowercase letter if you add -32, you will get it's capital equivalent.

After this, we have some constants labelled STACK_POSITIONS. Remember that function parameters are pushed onto the stack before function calls. These constants (prefixed with ST for clarity) define where in the stack we should expect to find each piece of data. The return address is at position 4 + %esp, the length of the buffer is at position 8 + %esp, and the address of the buffer is at position 12 + %esp. Using symbols for these numbers instead of the numbers themselves makes it easier to see what data is being used and moved.

Next comes the label `convert_to_upper`. This is the entry point of the function. The first two lines are our standard function lines to save the stack pointer. The next two lines

```
movl  ST_BUFFER(%ebp), %eax
```

Chapter 5. Dealing with Files

```
movl    ST_BUFFER_LEN(%ebp), %ebx
```

move the function parameters into the appropriate registers for use. Then, we load zero into `%edi`. What we are going to do is iterate through each byte of the buffer by loading from the location `%eax + %edi`, incrementing `%edi`, and repeating until `%edi` is equal to the buffer length stored in `%ebx`. The lines

```
cmpl    $0, %ebx
je      end_convert_loop
```

are just a sanity check to make sure that no one gave us a buffer of zero size. If they did, we just clean up and leave. Guarding against potential user and programming errors is an important task of a programmer. You can always specify that your function should not take a buffer of zero size, but it's even better to have the function check and have a reliable exit plan if it happens.

Now we start our loop. First, it moves a byte into `%cl`. The code for this is

```
movb    (%eax,%edi,1), %cl
```

It is using an indexed indirect addressing mode. It says to start at `%eax` and go `%edi` locations forward, with each location being 1 byte big. It takes the value found there, and put it in `%cl`. After this it checks to see if that value is in the range of lower-case *a* to lower-case *z*. To check the range, it simply checks to see if the letter is smaller than *a*. If it is, it can't be a lower-case letter. Likewise, if it is larger than *z*, it can't be a lower-case letter. So, in each of these cases, it simply moves on. If it is in the proper range, it then adds the uppercase conversion, and stores it back into the buffer.

Either way, it then goes to the next value by incrementing `%cl`. Next it checks to see if we are at the end of the buffer. If we are not at the end, we jump back to the beginning of the loop (the `convert_loop` label). If we are at the end, it simply continues on to the end of the function. Because we are modifying the buffer directly, we don't need to return anything to the calling program - the changes are

already in the buffer. The label `end_convert_loop` is not needed, but it's there so it's easy to see where the parts of the program are.

Now we know how the conversion process works. Now we need to figure out how to get the data in and out of the files.

Before reading and writing the files we must open them. The UNIX open system call is what handles this. It takes the following parameters:

- `%eax` contains the system call number as usual - 5 in this case.
- `%ebx` contains a pointer to a string that is the name of the file to open. The string must be terminated with the null character.
- `%ecx` contains the options used for opening the file. These tell Linux how to open the file. They can indicate things such as open for reading, open for writing, open for reading and writing, create if it doesn't exist, delete the file if it already exists, etc. We will not go into how to create the numbers for the options until the Section called *Truth, Falsehood, and Binary Numbers* in Chapter 10. For now, just trust the numbers we come up with.
- `%edx` contains the permissions that are used to open the file. This is used in case the file has to be created first, so Linux knows what permissions to create the file with. These are expressed in octal, just like regular UNIX permissions.⁶

After making the system call, the file descriptor of the newly-opened file is stored in `%eax`.

So, what files are we opening? In this example, we will be opening the files specified on the command-line. Fortunately, command-line parameters are already stored by Linux in an easy-to-access location, and are already null-terminated. When a Linux program begins, all pointers to command-line arguments are stored on the stack. The number of arguments is stored at `8(%esp)`, the name of the program is stored at `12(%esp)`, and the arguments are stored from `16(%esp)` on.

6. If you aren't familiar with UNIX permissions, just put `$0666` here. Don't forget the leading zero, as it means that the number is an octal number.

Chapter 5. Dealing with Files

In the C Programming language, this is referred to as the `argv` array, so we will refer to it that way in our program.

The first thing our program does is save the current stack position in `%ebp` and then reserve some space on the stack to store the file descriptors. After this, it starts opening files.

The first file the program opens is the input file, which is the first command-line argument. We do this by setting up the system call. We put the file name into `%ebx`, the read-only mode number into `%ecx`, the default mode of `$0666` into `%edx`, and the system call number into `%eax`. After the system call, the file is open and the file descriptor is stored in `%eax`.⁷ The file descriptor is then transferred to its appropriate place on the stack.

The same is then done for the output file, except that it is created with a write-only, create-if-doesn't-exist, truncate-if-does-exist mode. Its file descriptor is stored as well.

Now we get to the main part - the read/write loop. Basically, we will read fixed-size chunks of data from the input file, call our conversion function on it, and write it back to the output file. Although we are reading fixed-size chunks, the size of the chunks don't matter for this program - we are just operating on straight sequences of characters. We could read it in with as little or as large of chunks as we want, and it still would work properly.

The first part of the loop is to read the data. This uses the `read` system call. This call just takes a file descriptor to read from, a buffer to write into, and the size of the buffer (i.e. - the maximum number of bytes that could be written). The system call returns the number of bytes actually read, or end-of-file (the number 0).

7. Notice that we don't do any error checking on this. That is done just to keep the program simple. In normal programs, every system call should normally be checked for success or failure. In failure cases, `%eax` will hold an error code instead of a return value. Error codes are negative, so they can be detected by comparing `%eax` to zero and jumping if it is less than zero.

After reading a block, we check `%eax` for an end-of-file marker. If found, it exits the loop. Otherwise we keep on going.

After the data is read, the `convert_to_upper` function is called with the buffer we just read in and the number of characters read in the previous system call.

After this function executes, the buffer should be capitalized and ready to write out. The registers are then restored with what they had before.

Finally, we issue a `write` system call, which is exactly like the `read` system call, except that it moves the data from the buffer out to the file. Now we just go back to the beginning of the loop.

After the loop exits (remember, it exits if, after a read, it detects the end of the file), it simply closes its file descriptors and exits. The `close` system call just takes the file descriptor to close in `%ebx`.

The program is then finished!

Review

Know the Concepts

- Describe the lifecycle of a file descriptor.
- What are the standard file descriptors and what are they used for?
- What is a buffer?
- What is the difference between the `.data` section and the `.bss` section?
- What are the system calls related to reading and writing files?

Use the Concepts

- Modify the `toupper` program so that it reads from `STDIN` and writes to `STDOUT` instead of using the files on the command-line.
- Change the size of the buffer.
- Rewrite the program so that it uses storage in the `.bss` section rather than the stack to store the file descriptors.
- Write a program that will create a file called `hey now.txt` and write the words "Hey diddle diddle!" into it.

Going Further

- What difference does the size of the buffer make?
- What error results can be returned by each of these system calls?
- Make the program able to either operate on command-line arguments or use `STDIN` or `STDOUT` based on the number of command-line arguments specified by `ARGC`.
- Modify the program so that it checks the results of each system call, and prints out an error message to `STDOUT` when it occurs.